

PM_Collegio v1 — Documentazione Funzionale Completa

Versione: 1.1 | Data: 5 giugno 2026 | Autore: Giresse Kengne

1. Introduzione

PM_Collegio è un sistema desktop sviluppato in Java (Swing/JFrame) per la gestione delle prenotazioni di camere in collegi universitari. L'applicazione supporta uno scenario multi-committente ma nella versione attuale è ottimizzata per un singolo committente. Il database di supporto è MySQL (schema `pm_collegiov2`).

L'applicazione offre un'interfaccia grafica completa che guida l'operatore attraverso tutte le fasi operative: registrazione degli ospiti, assegnazione delle camere, check-in e check-out, generazione e gestione delle fatture, e supervisione amministrativa con controllo degli accessi basato su ruolo.

2. Architettura del Sistema

2.1 Stack Tecnologico

Componente	Tecnologia
Linguaggio	Java SE (JDK compatibile NetBeans IDE)
UI Framework	Java Swing / JFrame con AbsoluteLayout (<code>org.netbeans.lib.awtextra</code>)
Database	MySQL 8.x, schema <code>pm_collegiov2</code>
Driver JDBC	<code>com.mysql.cj.jdbc.Driver</code> (MySQL Connector/J)
Build tool	Apache Ant (<code>build.xml</code> generato da NetBeans)
Pattern architetturale	Monolitico piatto: ogni JFrame gestisce UI, logica applicativa e accesso al DB direttamente

2.2 Package e Struttura File

Tutto il codice risiede nel package `PM`. Ogni JFrame ha il suo file `.java` affiancato da un file `.form` NetBeans per la definizione visuale dell'interfaccia (ad eccezione di `fattura.java` e `gestioneFatture.java`, che sono pure Java senza `.form`).

I componenti aggiunti programmaticamente (fuori dal blocco `GEN-BEGIN / GEN-END`) richiedono `setComponentZOrder(comp, 0)` per apparire sopra l'immagine di sfondo.

2.3 Flusso di Navigazione Principale

`loading1` (splash 1 sec) → `loading` (progress bar 0-100%) → `login` → `homeAdmin` → [moduli per ruolo]

Ogni modulo è una finestra JFrame indipendente che si apre sopra la `homeAdmin`. La chiusura di un modulo non chiude la `homeAdmin`. La sessione è gestita tramite la classe statica `Session.java`, che mantiene in memoria i dati dell'utente loggato per tutta la durata della sessione applicativa.

3. Ruoli e Permessi

Il sistema implementa quattro ruoli distinti, definiti nella tabella `Mansione` (campo `role_type`). La visibilità dei pulsanti nella dashboard è controllata dal metodo `configureButtonsForRole()` in `homeAdmin.java`.

Ruolo	Codice	ManageUser	ManageReservation	ManageRoom	ManageTenant	CheckIn / CheckOut	AccessLogs
Admin Sistema	AS	SI	SI	SI	SI	SI	SI
Admin Committente	AC	SI	SI	SI	NO	SI	SI
Receptionist	AR	SI	SI	NO	NO	SI	NO
Utente / Cliente	U	Solo propria utenza *	Solo proprie **	NO	NO	SI	NO

* ManageUser per ruolo U: il pulsante è visibile ma mostra solo i propri dati, senza tabella di ricerca né possibilità di aggiungere utenti.

** ManageReservation per ruolo U: visibile ma filtrato. La tabella mostra solo le proprie prenotazioni; la ricerca per ID è ristretta alle proprie; il pulsante Update è nascosto. Tramite il pulsante Fatture accede a GestioneFatture filtrata sulle proprie fatture, con possibilità di pagare e annullare.

4. Moduli dell'Applicazione

4.1 Avvio e Schermata di Caricamento

All'avvio vengono mostrate due schermate in sequenza.

- **loading1.java**: splash screen con immagine di sfondo, rimane visibile per circa un secondo.
- **loading.java**: progress bar animata con messaggi di stato progressivi: *Loading Modules...* al 25%, *Connecting Database...* al 50%, *Launching Application...* al 95%. Al raggiungimento del 100%, la schermata di login si apre automaticamente.

4.2 Login e Autenticazione (`login.java`)

La schermata di login è il punto di ingresso per tutti gli utenti. L'autenticazione avviene tramite due query sequenziali: la prima verifica email e password, la seconda recupera i dati completi (nome, cognome, committente, ruolo). Solo gli utenti con stato `attivo` possono accedere.

Dopo un login riuscito, il sistema esegue in sequenza:

1. Registra l'accesso in `AccessLog` (`user_id` , `ip='127.0.0.1'` , `note='Desktop App'`) e salva il `log_id` in `Session.logId`.
2. Crea un token di sessione UUID a 64 caratteri nella tabella `sessions` con scadenza +8 ore.
3. Carica tutti i dati utente nell'oggetto statico `Session` (`userCounter` , `email` , `nome` , `cognome` , `roleType` , `committenteId` , `sessionToken`).
4. Apre `homeAdmin` e chiude la schermata di login.

Funzionalità UI: validazione email live con icona verde/rossa, toggle visibilità password, link a registrazione, link a recupero password, supporto tasto Enter per navigare tra i campi.

4.3 Registrazione Utente (`registration.java`)

Permette la creazione di un nuovo account. I campi obbligatori sono: nome, cognome, email (univoca), genere, password, indirizzo, telefono (10 cifre esatte), domanda di sicurezza (6 opzioni predefinite) e risposta. La validazione di email e telefono avviene in tempo reale con feedback visivo.

Il nuovo utente viene creato con stato `attesa` (deve essere attivato da un admin), ruolo 4 (cliente), `committente_id=1`. L' `user_counter` è generato automaticamente come `'XXX'` + nome in maiuscolo (es. XXXMARIO).

4.4 Recupero Password (`passWord.java`)

Permette il reset della password in autonomia tramite domanda di sicurezza.

Flusso: inserimento email → click *Search* → domanda di sicurezza caricata → inserimento risposta e nuova password → click *Confirm*. Il sistema verifica la correttezza della risposta prima di aggiornare la password. In caso di risposta errata, nessuna modifica viene apportata.

4.5 Home Page - Dashboard (`homeAdmin.java`)

Punto centrale dell'applicazione, funge da dashboard unica per tutti i ruoli. Il menu laterale destro mostra solo i pulsanti consentiti per il ruolo corrente (`Session.roleType`). In alto a destra è sempre visibile il nome e cognome dell'utente loggato con icona identificativa.

Layout verticale dei pulsanti (tutti a `x=1360` , larghezza=250, altezza=80):

Pulsante	Posizione (y)	Visibilità
ManageUser	50	Tutti i ruoli
ManageReservation	165	Tutti i ruoli (filtrato per U)
ManageRoom	280	AS, AC
ManageTenant	395	Solo AS
CheckIn	510	Tutti i ruoli
CheckOut	625	Tutti i ruoli
Access Logs	740	AS, AC
Logout (rosso)	855	Tutti i ruoli

Il logout registra il timestamp di uscita in `AccessLog` , invalida il token nella tabella `sessions` , azzerà tutti i campi dell'oggetto `Session` e torna alla schermata di login. La X di chiusura compie le stesse operazioni ma termina il processo invece di tornare al login.

4.6 Check-In (`checkIn.java`)

Gestisce l'assegnazione di una camera a un ospite e la creazione della prenotazione. All'apertura, la combo box camere è popolata automaticamente con le sole camere disponibili. La data di check-in è quella odierna (non modificabile).

Flusso operativo:

1. Inserimento email cliente e click *Search*: dati utente caricati automaticamente e resi non modificabili.
2. Selezione camera dalla combo: tipo, letto e prezzo si aggiornano in tempo reale (`ItemStateChanged`).
3. Click *Allote* per confermare.

Alla conferma, una transazione atomica esegue:

- `INSERT Reservation` (status= `attiva` , check_in=oggi, note=null)
- `INSERT Fattura` (importo=0, stato= `in attesa` — l'importo definitivo viene calcolato al checkout)
- `UPDATE Room` stato= `occupata`

In caso di successo: messaggio con ID prenotazione (formato `P0XXX`), form pulito per nuovo check-in immediato.

Bug risolti:

- `ItemStateChanged` : gestisce correttamente l'evento `DESELECTED` durante `removeAllItems()` tramite `guard SELECTED + index ≥ 0` , evitando la `NullPointerException` che impediva nuovi check-in consecutivi.
- Connessione DB: refactoring a pattern connection-per-operation con `try-with-resources` . Ogni metodo apre e chiude la propria connessione in modo automatico e garantito; nessuna connessione rimane aperta per tutta la durata della finestra.

4.7 Check-Out e Generazione Fattura (`checkout.java`)

Gestisce la chiusura di una prenotazione con calcolo automatico dell'importo. La data di checkout è quella odierna.

Flusso operativo:

1. Inserimento ID prenotazione (formato `P0XXX`) e click *Search*: caricamento automatico di nome, telefono, email, camera, data check-in, giorni e importo totale.
2. Verifica dati e click *CheckOut* per confermare.

Calcolo importo: giorni = differenza tra data checkout e data check-in, minimo 1 giorno. Importo = prezzo_notte × giorni.

La transazione atomica esegue:

- `UPDATE Reservation` (check_out=oggi, status= `completata` , note= `Camera libera` , giorni)
- `UPDATE Room` stato= `disponibile`
- Upsert Fattura: se esiste una fattura per quella prenotazione, aggiorna importo e data; altrimenti crea nuova fattura con stato= `in attesa` .

Al termine, si apre automaticamente la finestra di visualizzazione della fattura.

Pulizia UI: il form di checkout aveva una `JTable` in basso ereditata dal designer ma mai utilizzata dal codice. La tabella e il relativo `JScrollPane` sono stati rimossi sia dal `.java` sia dal `.form` per chiarezza.

4.8 Visualizzazione Fattura (`fattura.java`)

Pannello riepilogativo che mostra tutti i dati della fattura: numero fattura, prenotazione (`P0XXX`), nome cliente, numero camera, data emissione, stato e importo evidenziato in verde. Si apre automaticamente al termine del checkout.

Azioni disponibili (attive solo se la fattura è in stato `in attesa` e la prenotazione correlata è in stato `completata` o `cancellata`):

- **Paga:** apre un dialog con la lista dei metodi di pagamento (da tabella `MetodoPagamento`). Alla selezione e conferma: `INSERT in Pagamento` (metodo, importo, data), `UPDATE Fattura` stato= `pagato` . La finestra si chiude automaticamente.
- **Annulla:** dopo conferma, `UPDATE Fattura` stato= `non pagato` . La finestra si chiude automaticamente.

Regola di pagabilità: non è sufficiente che la fattura sia in stato `in attesa` : finché la prenotazione è ancora `attiva` (check-in fatto, check-out non ancora eseguito) i bottoni Paga e Annulla restano disabilitati. La query di caricamento della fattura include il JOIN su `Reservation` per recuperare lo stato corrente.

4.9 Gestione Fatture (`gestioneFatture.java`)

Accessibile tramite il pulsante *Fatture* in *ManageReservation*. Mostra la tabella completa di tutte le fatture (ordinate per ID decrescente) con colonne: ID, prenotazione, cliente, camera, importo, data, stato, **stato prenotazione**. La colonna *Stato Prenot.* permette di vedere a colpo d'occhio se la fattura è pagabile o se la prenotazione è ancora aperta.

Click su riga: i pulsanti *Paga* e *Annulla* si attivano solo se:

- lo stato della fattura è `in attesa` o `non pagato` , E
- lo stato della prenotazione correlata è `completata` o `cancellata` .

Per fatture in stato `pagato` o con prenotazione ancora `attiva` , i bottoni rimangono disabilitati. Il meccanismo di pagamento è identico a quello della finestra fattura. Dopo ogni azione la tabella si ricarica automaticamente.

4.10 Gestione Prenotazioni (`manageReservation.java`)

Permette la ricerca, visualizzazione e modifica delle prenotazioni esistenti. All'apertura tutti i campi sono vuoti.

Ricerca per ID prenotazione (formato `P0XXX`) : caricamento di tutti i dati associati. Modifica: aggiornamento date e note. La transazione include `UPDATE Reservation` e `INSERT in StoricoPrenotazioni` con le date precedenti e nuove (audit completo).

La tabella in basso mostra tutte le prenotazioni con colonne: ID, User, Committente, CheckIn, CheckOut, Status, Room, Note, Giorni, Totale. Il pulsante *Fatture* (posizionato a sinistra di *Update*) apre direttamente la schermata `gestioneFatture` .

Click su riga della tabella: al click su una qualsiasi riga, tutti i campi del form sopra (id, user, committente, check-in, check-out, status, room, note, giorni, totale) vengono popolati istantaneamente con i valori della riga selezionata. L'operatore non deve digitare l'ID e cliccare Search: basta selezionare la prenotazione dalla tabella. Lo stato viene scritto nel `ppdTextField` sotto la label "Status".

4.11 Gestione Camere (`manageRoom.java`)

Accessibile ad AS e AC. Gestione completa del parco camere:

- **Visualizzazione:** tabella con tipo, letto, prezzo, stato.
- **Aggiunta:** form con numero stanza, tipo, letto, prezzo. ID camera auto-generato come `MAX+1` .
- **Modifica:** click singolo su riga carica il form.
- **Eliminazione:** doppio click con conferma.

Le camere con stato `occupata` non sono modificabili né eliminabili.

4.12 Gestione Utenti (`manageUser.java`)

Comportamento adattivo per ruolo:

- **AS / AC / AR:** visualizzazione tabella completa (nome, email, password, risposta sicurezza, stato). Ricerca per email, modifica e aggiunta utenti. Campi modificabili: nome, cognome, password, telefono, stato, domanda/risposta sicurezza, genere.
- **U (Cliente):** tabella e controlli di ricerca/aggiunta nascosti. Mostra solo il form con i propri dati precompilati (email non modificabile). Può aggiornare i propri dati personali.

4.13 Gestione Committenti (`manageTenant.java`)

Accessibile solo all'AS. Gestisce i committenti (collegi) presenti nel sistema.

Tabella: ID, ragione sociale, admin gestore, email. Form per insert/update: ragione sociale, email, telefono, gestore (selezione tra admin attivi), indirizzo.

Comportamento pulsanti: *Add* attivo di default; click su riga → *Add* disabilitato, *Update* abilitato; *Clear* → ripristino stato iniziale.

4.14 Log Accessi (`logs.java`)

Accessibile ad AS e AC. Tabella di sola lettura con lo storico degli accessi: nome utente, data/ora login, data/ora logout, IP, ruolo. Ordinata per login decrescente. Il campo `logout_time` è `NULL` per le sessioni ancora aperte.

5. Gestione della Sessione e Sicurezza

La sessione è gestita tramite la classe statica `Session.java`, che mantiene in memoria i dati dell'utente durante tutta la sessione applicativa. Non persiste su disco: si azzerà alla chiusura dell'applicazione o al logout.

Ogni accesso viene tracciato nella tabella `AccessLog` con `user_id`, `ip_address`, `note` e timestamp automatico di login. Il timestamp di logout viene aggiornato al momento dell'uscita (pulsante Logout o chiusura finestra), garantendo un audit trail completo.

Contenuto dell'oggetto Session

Campo	Tipo	Descrizione
<code>userCounter</code>	String	ID utente (es. XXXMARIO)
<code>email</code> / <code>nome</code> / <code>cognome</code>	String	Dati anagrafici dell'utente loggato
<code>logId</code>	int	ID della riga <code>AccessLog</code> corrente
<code>roleId</code> / <code>roleNome</code> / <code>roleType</code>	int / String / String	Dati ruolo (<code>AS</code> , <code>AC</code> , <code>AR</code> , <code>U</code>)
<code>committenteId</code>	int	FK Committente di appartenenza
<code>sessionToken</code>	String (64 char)	UUID per validazione lato DB (tabella <code>sessions</code>)

Tabella sessions (DB)

Colonna	Descrizione
session_id	Chiave primaria
user_id	FK → User
committente_id	FK → Committente
role_id	FK → Mansione
token	CHAR(64), UUID generato al login
expires	DATETIME, scadenza = login + 8 ore
created_at	Timestamp creazione

Invalidazione: al logout o chiusura finestra, viene eseguito `UPDATE sessions SET expires = NOW()` (invalidazione anticipata) e tutti i campi dell'oggetto `Session` vengono azzerati.

6. Validazioni e Regole di Business

6.1 Validazioni di Input

- **Email:** validazione con regex live durante la digitazione, icona verde (valida) o rossa (non valida).
- **Telefono:** esattamente 10 cifre numeriche, validazione live con feedback colorato.
- **Campi obbligatori:** tutti verificati prima del submit; messaggi di errore specifici per campo mancante.
- **Prezzo camera:** parsing Double, errore se non numerico.
- **Unicità email:** verificata su DB prima di inserire un nuovo utente.

6.2 Regole di Business Principali

- Nuovo utente registrato → stato `attesa`, deve essere attivato da un admin.
- Login consentito solo a utenti con stato `attivo`.
- Camera `occupata`: non modificabile né eliminabile.
- Giorni soggiorno: minimo 1 (anche se check-in e check-out coincidono).
- Importo fattura = `prezzo_per_notte` × `numero_giorni`, calcolato automaticamente al checkout.
- Fattura: upsert al checkout (aggiorna se esiste, crea se non esiste).
- Paga / Annulla fattura: abilitati solo se la fattura è in stato `in attesa` o `non pagato` e la prenotazione correlata è in stato `completata` o `cancellata`.
- Ogni modifica a una prenotazione genera una riga di audit in `StoricoPrenotazioni`.
- Sessione: scade automaticamente dopo 8 ore, invalidata esplicitamente al logout.

7. Flusso Operativo Tipico (dalla A alla Z)

1. **AVVIO:** splash screen (1 sec) → progress bar → schermata login.
2. **LOGIN:** inserimento credenziali → verifica DB → caricamento `Session` → apertura `homeAdmin`.
3. **REGISTRAZIONE** (nuovo ospite): `manageUser` → `Add` → inserimento dati → stato `attesa` → attivazione da admin.
4. **CHECK-IN:** `checkIn` → ricerca ospite per email → selezione camera disponibile → `Allote` → prenotazione creata (formato `P0XXX`).
5. **GESTIONE:** `manageRiservation` → modifica date/note se necessario → `StoricoPrenotazioni` aggiornato.
6. **CHECK-OUT:** `checkOut` → inserimento ID prenotazione → dati caricati → importo calcolato → conferma → finestra fattura aperta.
7. **PAGAMENTO:** `fattura` → selezione metodo → conferma → stato `pagato` registrato.
8. **CONSULTAZIONE:** `gestioneFatture` per panoramica completa delle fatture; `logs` per audit degli accessi.
9. **LOGOUT:** pulsante `Logout` → timestamp uscita registrato in `AccessLog` → sessione invalidata → ritorno alla schermata di login.

8. Note Tecniche e Limitazioni

8.1 Configurazione Database

La connessione al database è centralizzata tramite la classe `DBConfig.java` e il file di configurazione `db.properties`, caricato dal classpath all'avvio. Nessuna stringa di connessione è hardcoded nel codice sorgente: tutte le classi che accedono al DB chiamano esclusivamente `DBConfig.getConnection()`.

Componente	Responsabilità
<code>db.properties</code>	File di configurazione esterno (non versionato). Contiene <code>db.url</code> , <code>db.user</code> , <code>db.password</code> . Caricato una sola volta via <code>ClassLoader</code> .
<code>DBConfig.java</code>	Classe di utilità con blocco statico che legge <code>db.properties</code> e espone il metodo <code>getConnection(): Connection</code> .

Parametro	Valore (da <code>db.properties</code>)
<code>db.url</code>	<code>jdbc:mysql://localhost:3306/pm_collegiov2</code>
<code>db.user</code>	<code>root</code>
<code>db.password</code>	<code>Giresse98!</code>

8.2 Typo e Anomalie Note

File / Elemento	Problema	Impatto
<code>home.java</code>	Presente nel codebase ma non più usata (rimpiazzata da <code>homeAdmin</code> per tutti i ruoli)	Dead code — può essere rimosso in futuro
<code>checkIn.java</code> / <code>manageUser.java</code>	<code>committente_id</code> hardcoded a 1 negli INSERT	Funziona per installazione a committente singolo, da parametrizzare in futuro

8.3 Limitazioni Funzionali

- **StoricoPrenotazioni:** popolata al salvataggio delle modifiche prenotazione, ma non ancora visualizzata in UI.
- **Pagamenti:** simulati (nessun gateway reale). I metodi di pagamento sono letti dalla tabella `MetodoPagamento` ma la transazione finanziaria effettiva non avviene.
- **IP registrato:** sempre `127.0.0.1` (hardcoded), poiché è un'applicazione desktop locale.
- **Multi-committenza:** lo schema DB la supporta, ma l'applicazione è sviluppata e testata per un singolo committente (`committente_id=1`).

9. Storico modifiche

Versione	Data	Modifiche
1.0	26 maggio 2026	Prima versione della documentazione funzionale completa.
1.1	5 giugno 2026	• Rinominato <code>loadind1.java</code> in <code>loading1.java</code> (typo corretto). • Rimossa la <code>JTable</code> inutilizzata dal form di <code>checkOut.java</code>. • Aggiunto in <code>manageRiservation.java</code>: click su riga della tabella popola istantaneamente il form sopra. • Nuova regola di pagabilità fattura: i bottoni Paga e Annulla si abilitano solo se la fattura è in stato <code>in attesa</code> / <code>non pagato</code> e la prenotazione correlata è in stato <code>completata</code> / <code>cancellata</code> (sia in <code>fattura.java</code> sia in <code>gestioneFatture.java</code>). • Aggiunta colonna <i>Stato Prenot.</i> alla tabella di <code>gestioneFatture.java</code> per dare feedback visivo sulla pagabilità.